Implementação DevOps — Aprova.AI

GWBR Technologies · Documento gerado a partir de `docs/IMPLEMENTACAO-DEVOPS.md` em 30/10/2026

Guia para a equipe de infraestrutura, integração e entrega contínuas e operação. Reúne tudo o que é preciso saber para colocar o Aprova.AI no ar com recursos gratuitos: topologia, provisionamento na Oracle Cloud, Docker Compose, Redis e PostgreSQL, DNS e HTTPS com o domínio existente, segredos, CI/CD no GitHub, builds móveis com o EAS, observabilidade, backups, migração para o Render, custos por mil usuários e runbooks. Deriva de `docs/ARQUITETURA.md`; em caso de divergência, o documento de arquitetura prevalece.

Sumário

1. Base comum a todas as equipes
2. Ponto de partida e responsabilidades do DevOps
3. Topologia de produção e dimensionamento
4. Provisionamento na Oracle Cloud
5. Preparação e endurecimento do servidor
6. Docker Compose de produção
7. Redis e PostgreSQL: configuração exigida
8. DNS, domínio e HTTPS
9. Segredos e configuração
10. Integração contínua
11. Entrega contínua do backend
12. Pipeline móvel e lojas
13. Observabilidade e alertas
14. Backups e recuperação
15. Ambientes: desenvolvimento local e staging
16. Escala e migração para o Render
17. Custos e limites gratuitos por mil usuários
18. Runbooks
19. Ordem de implementação e critérios de aceite
20. Checklist de go-live e pendências

1. Base comum a todas as equipes

Este capítulo é idêntico nos três guias de implementação (Front-End, Back-End e DevOps). Ele existe para que nenhuma equipe dependa de ler o guia da outra para entender o produto, as decisões, o recurso Premium, os contratos e os custos.

1.1 O produto e o objetivo da arquitetura

O Aprova.AI é um aplicativo para Android e iOS, feito com Expo e TypeScript, para estudo para o Enem, a Fuvest e outros vestibulares brasileiros, com correção e apoio por IA (GPT 6.1 - Terra). O objetivo técnico é duplo: o app nunca trava e nenhuma atividade em segundo plano se perde.

Para isso o sistema separa o que o aluno faz na tela (comandos, que mudam estado e respondem na hora) do que o sistema processa nos bastidores (eventos, fila e Workers, onde mora a IA) e do que o aluno consulta (modelos de leitura, eventualmente consistentes). Quando o aluno envia uma redação, a API responde `202 Accepted` em milissegundos, o app mostra “Redação recebida com sucesso!” e um Worker corrige a redação sem risco de timeout da API.

1.2 Estado de partida verificado em 30/10/2026

Item	Valor verificado
Expo, React Native, React	`expo ~57.0.24`, `react-native 0.86.3`, `react 19.2.3`
TypeScript	`~6.0.3`, modo estrito (`tsconfig.json` estende `expo/tsconfig.base`)
Entrada do app	`index.ts` na raiz (`registerRootComponent`), que importa `src/app/App.tsx` (uma tela vazia)
Scripts do `package.json`	Somente `start`, `android`, `ios` e `web`. **Não existem** scripts de lint, verificação de tipos nem testes
Backend e infraestrutura	Não há código. Existem somente `README.md` de camada em `backend/src/*`, `contracts/`, `src/features/` e `src/core/`
Repositório	`GWBR-Technologies/Aprova.AI`, **público**, branches `master` e `development`
`.gitignore`	Cobre `.env*.local`, mas **não** `.env`; deve ser corrigido antes do primeiro uso de variáveis de ambiente
Dependências de runtime hoje	Apenas `expo`, `expo-status-bar`, `react` e `react-native`

Duas observações que importam para todos. Primeiro, as regras do projeto citam `npm run lint` como o comando de qualidade, mas esse script não existe no `package.json`; a proposta P8 (§1.5) cria o script, e até lá nenhuma equipe pode afirmar que o lint passou. Segundo, como o repositório é público, tudo o que for versionado, inclusive este guia e os logs da integração contínua, é visível a qualquer pessoa: segredos, chaves e dados de alunos nunca entram nele.

1.3 Regras do projeto que valem para todos

- Baseie decisões no código e na configuração existentes, não em suposições sobre o produto. Declare uma suposição somente quando ela afetar o resultado; se a escolha alterar escopo, dados de medição ou risco para o usuário, peça direção.
- Mudanças enxutas: altere só os arquivos necessários, não refatore código vizinho, remova apenas o que a própria mudança tornou obsoleto e não crie abstração para um único uso.
- Não adicione dependência sem aprovação explícita. As propostas do §1.5 estão listadas justamente para serem aprovadas uma a uma.
- Não invente dados de produto: banco de questões, notas, correções, rubricas, preços e textos de marketing vêm do produto. Quando faltar o requisito, pare e peça.
- Siga os padrões existentes (React Native, componentes temáticos, arquivos específicos de plataforma) e os tokens de `src/constants/theme.ts`, que nasce no primeiro uso.
- Comentários explicam decisões, restrições e comportamento não óbvio; não há comentários TODO.
- Informe sempre os comandos executados, os resultados, as limitações e as plataformas que não puderam ser verificadas. Não afirme que algo funciona sem evidência da validação correspondente.

- Não leia, imprima, altere nem versione segredos, `.env` ou `.npmrc`. Não use comandos destrutivos (`rm -rf`, `git clean`, `git reset --hard`, push forçado).
- Commits seguem o padrão `(tipo): Frase no passado`, por exemplo `(docs): Adicionado README.md completo e estruturado desta fase inicial`, com um commit por assunto. Mensagens de commit e descrições de PR não mencionam ferramentas de assistência nem coautoria.
- Qualquer mudança de decisão ou de estrutura atualiza `docs/ARQUITETURA.md`, o `README.md`, os guias e os PDFs, na ordem do §13 do documento de arquitetura.

1.4 Decisões já tomadas

Id	Decisão
D1	Arquitetura geral: Event-Driven Architecture + CQRS.
D2	Fila: BullMQ + Redis, atrás de uma porta única (`QueuePort`); SQS+SNS e RabbitMQ são alternativas.
D3	Comandos respondem de imediato (`202 Accepted`); o trabalho pesado roda em Workers.
D4	O app mantém um outbox local e envia cada comando com chave de idempotência (`commandId`).
D5	O servidor grava estado e mensagem de saída na mesma transação (outbox transacional) e um relay publica na fila.
D6	A IA só é chamada por Workers, atrás de `AiPort`.
D7	Repositório único: app Expo na raiz, `backend/` e `contracts/`, sem workspaces nesta fase.
D8	O lado de leitura é eventualmente consistente; a interface mostra estados explícitos.
D9	Correção de redação por IA é exclusiva do plano `PREMIUM`, com 4 correções por mês (§1.6).
D10	Todo recurso precisa de opção gratuita; custo só com aprovação explícita, exceto a IA (§1.8).
D11	Hospedagem inicial na Oracle Cloud Always Free com Docker Compose; migração planejada para o Render (pago).
D12	Lançamento em Android e iOS; as contas de desenvolvedor da Apple e do Google já existem e as licenças ficam fora do orçamento.
D13	O domínio existente (Hostinger) serve a API por um subdomínio `api.` apontado para o servidor.

1.5 Propostas que aguardam aprovação

Estas propostas são o caminho gratuito que os guias assumem. Cada uma só vira decisão depois de aprovada; até lá, ninguém instala o pacote nem cria o recurso. Todos os pacotes citados são de código aberto (licenças verificadas no registro do npm em 30/10/2026).

Id	Proposta	Custo
P1	Autenticação: login com Google e com Apple (OIDC); a API valida o token do provedor e emite JWT de acesso curto e refresh token rotativo. Sem senha e sem e-mail transacional.	US$ 0
P2	Banco: PostgreSQL 18 (suporte até 14/11/2030) em uma instância, com os esquemas `write` e `read` separados.	US$ 0
P3	Backend: Node.js 24 LTS executando TypeScript nativamente (sem etapa de build), Fastify 5, Drizzle ORM com `pg`, BullMQ 6 com `ioredis`, `zod` 4 (só no backend), `pino`, `prom-client` e o SDK `openai`.	US$ 0 (exceto a IA)
P4	Canal de eventos para o app: consulta periódica com intervalo crescente, com push do Expo como aviso sem dados pessoais.	US$ 0

Id	Proposta	Custo
P5	App: Expo Router, TanStack Query, `expo-sqlite`, `expo-secure-store`, `expo-notifications`, `@react-native-community/netinfo`, `expo-crypto`, `@react-native-google-signin/google-signin` e `expo-apple-authentication`.	US$ 0
P6	`contracts/` continua só com tipos; o backend valida com `zod` e garante a compatibilidade com os tipos em tempo de compilação. D7 permanece como está.	US$ 0
P7	Testes: Vitest no backend; `jest-expo` e `@testing-library/react-native` no app.	US$ 0
P8	Scripts `lint` e `typecheck` na raiz e em `backend/` (ESLint com `eslint-config-expo` no app).	US$ 0
P9	Observabilidade: Sentry (plano gratuito) para erros e Grafana Cloud (plano gratuito) para métricas e logs.	US$ 0
P10	Entrega: GitHub Actions (repositório público), GitHub Container Registry e EAS Build no plano gratuito.	US$ 0

A coluna de custo vale enquanto o uso ficar dentro dos limites gratuitos do §1.8. Se algum limite for ultrapassado, a equipe responsável avisa antes, com o valor estimado.

1.6 Recurso Premium e cota de correção de redação (D9)

Esta é uma regra de produto que atravessa as três equipes.

Regra. Apenas usuários com plano `PREMIUM` ativo podem enviar redação para correção por IA, e cada um tem direito a 4 correções por mês. A regra é aplicada pelo servidor no comando `SubmitEssay`; o app apenas reflete o resultado.

Premissas a confirmar com o produto (nenhuma foi definida na conversa de arquitetura):

- A janela da cota é o mês corrido (do dia 1 às 23h59 do último dia) no fuso `America/Sao_Paulo`, e não 30 dias móveis.
- O plano que não é Premium chama-se `FREE` nos contratos; o nome de exibição é decisão do produto.
- Uma correção já aceita é concluída mesmo que o usuário perca o Premium durante o processamento; novos envios passam a ser rejeitados.
- A cota consumida pertence ao mês em que a redação foi aceita, mesmo que o Worker termine no mês seguinte.

Como a cota é consumida. A reserva e a gravação da redação ocorrem na mesma transação do comando. O servidor mantém um contador por usuário e por mês e o incrementa com um `UPDATE` condicional que só tem efeito se o valor atual for menor que 4. Se nenhuma linha for afetada, a cota acabou. Isso é atômico mesmo com envios simultâneos em dois aparelhos.

Situação	Resultado	Cota
Usuário sem plano Premium envia redação	`403` e `PREMIUM_REQUIRED`; o app mostra a oferta do Premium. Nada é gravado.	Não consome
Premium com a cota esgotada	`403` e `ESSAY_QUOTA_EXCEEDED` com `limit`, `used` e `resetsAt`. Nada é gravado.	Não consome
Premium com cota disponível	`202`; a redação e a mensagem de saída são gravadas.	Consome 1

Situação	Resultado	Cota
Mesmo `commandId` reenviado (rede ruim)	`202` idêntico ao primeiro.	Não consome de novo
Correção termina em falha permanente (`EssayCorrectionFailed`)	O app mostra a falha e a ação de reenviar.	Devolve 1, na mesma transação que grava a falha
Falha transitória que acaba corrigindo após novas tentativas	`EssayCorrected`.	Mantém o 1 consumido
Virada de mês com a correção em andamento	A correção conclui normalmente.	Fica no mês da submissão

O que a interface precisa mostrar. Para o usuário Premium, quantas correções restam no mês e quando renovam (`used`, `limit` e `resetsAt` vêm da consulta de direitos, §1.7). Para quem não é Premium, a oferta do Premium no ponto de envio da redação; o texto e o preço dependem do produto e não são inventados aqui. Em ambos os casos a informação vem do servidor, nunca de contagem local.

Testes exigidos para esta regra (back-end, com apoio do front-end nas telas): usuário sem Premium; Premium com 0, 3 e 4 usos; duas submissões simultâneas com 3 usos (apenas uma passa); reenvio do mesmo `commandId`; falha permanente devolvendo a cota; virada de mês em `America/Sao_Paulo` (por exemplo 31/10 às 23h59 e 01/11 às 00h00 no horário de Brasília); e perda do Premium durante o processamento.

Em aberto (§1.8 e §14 do documento de arquitetura). Como o usuário se torna Premium (assinatura, cobrança, preço, período de teste) ainda não foi decidido. A arquitetura isola isso atrás da `EntitlementPort`: o backend só pergunta “qual é o plano deste usuário agora?”. Enquanto a fonte real não existir, nenhuma equipe deve criar um mecanismo de concessão de Premium por conta própria.

1.7 Contratos entre o app e o backend

Todos os contratos moram em `contracts/` como **tipos** (sem lógica e sem código executável) e são a única fonte de verdade do formato. Um contrato novo só entra com aprovação.

Comandos. `POST /v1/commands/<nome-em-kebab-case>` com o corpo abaixo. O sucesso é sempre `202`.

```
// contracts/commands/command-envelope.ts (somente tipos)
export type CommandEnvelope<TPayload> = {
  commandId: string; // UUID v4 gerado no app; é a chave de idempotência
  issuedAt: string;  // ISO 8601 em UTC
  payload: TPayload;
};

export type CommandAccepted = {
  commandId: string;
  status: 'accepted';
  receivedAt: string; // ISO 8601 em UTC
};

// contracts/commands/submit-essay.command.ts
export type SubmitEssayPayload = {
  essayId: string; // UUID gerado no app
  text: string;
```

```
  // Tema e limites de tamanho do texto são definidos com o produto (§14).
};
```

Comandos iniciais (§5 do documento de arquitetura): `AnswerQuestion`, `RecordRoutineProgress`, `SubmitEssay` e `CompleteMockExam`. Os campos de negócio de cada payload (identificador da questão, alternativa, tempo, duração e assim por diante) são definidos com o produto antes da implementação; nenhuma equipe os inventa.

Consultas. `GET /v1/...`; toda resposta traz `asOf` (ISO 8601 em UTC) com o instante da última projeção aplicada.

Consulta	Rota	Resposta (resumo)
`GetEssay`	`GET /v1/essays/{essayId}`	`status` (`processing`, `completed`, `failed`), `result` quando concluída, `failure` quando falhou, `asOf`; `404` enquanto a projeção ainda não foi aplicada
`GetEssayList` (proposta, precisa de aprovação)	`GET /v1/me/essays`	Redações do usuário com `status` e data, paginadas, `asOf`
`GetEntitlements`	`GET /v1/me/entitlements`	`plan` (`FREE` ou `PREMIUM`) e `essayQuota` (`limit`, `used`, `resetsAt`) ou `null` quando não há direito, `asOf`
`GetPerformanceSummary`	`GET /v1/me/performance`	Estatísticas de desempenho do read store, `asOf`
`GetMockExamHistory`	`GET /v1/me/mock-exams`	Histórico de simulados, `asOf`
`GetMockExamResult`	`GET /v1/mock-exams/{mockExamId}`	`status` e resultado consolidado, `asOf`

O formato do `result` da redação depende da rubrica (em aberto) e não é definido aqui. O estado `received` do app é do **outbox local** (a API respondeu `202`); no servidor, uma redação aceita já está `processing`, e logo depois do `202` a consulta pode devolver `404` porque a projeção é assíncrona. O app trata esse `404` transitório como “em processamento”.

Erros. Toda resposta de erro tem o mesmo corpo, que nunca expõe detalhes internos:

```
export type ApiError = {
  code: ErrorCode;
  message: string;        // texto compreensível para o aluno
  correlationId: string;  // igual ao commandId quando houver
  details?: Record<string, unknown>;
};
```

Status	`code`	Quando	O app deve
400	`VALIDATION_FAILED`	Corpo, parâmetro ou cabeçalho inválido	Marcar o comando como falho e mostrar a correção necessária
401	`UNAUTHENTICATED`	Token ausente, inválido ou expirado	Renovar o token; se falhar, pedir novo login

Status	code	Quando	O app deve
403	`PREMIUM_REQUIRED`	Plano sem direito à correção	Mostrar a oferta do Premium; não reenviar
403	`ESSAY_QUOTA_EXCEEDED`	Cota mensal esgotada (`limit` , `used` , `resetsAt`)	Informar quando renova; não reenviar
409	`COMMAND_ID_CONFLICT`	Mesmo `commandId` com payload diferente	Tratar como bug; registrar e não reenviar
429	`RATE_LIMITED`	Excesso de requisições	Reenviar depois de `Retry-After`
500	`INTERNAL_ERROR`	Falha inesperada	Reenviar com intervalo crescente
503	`SERVICE_UNAVAILABLE`	Dependência fora do ar	Reenviar com intervalo crescente

Regra de reenvio do outbox: reenviar erros de rede, `408` , `429` e `5xx` ; nunca reenviar os demais `4xx` , que são permanentes e terminam o comando como `failed` .

Autenticação. `POST /v1/auth/login` (`{ provider, idToken }` , com `provider` igual a `google` ou `apple`), `POST /v1/auth/refresh` e `POST /v1/auth/logout` são endpoints HTTP síncronos, fora do modelo de comandos, porque devolvem tokens e não `202` . O login devolve `{ accessToken, accessTokenExpiresAt, refreshToken }` . Dependem da proposta P1.

Comando técnico proposto. `RegisterPushToken` (idempotente por token) só existe se a proposta P4 for aprovada; não faz parte do conjunto inicial.

Aviso por push. O push do Expo é só um aviso para o app consultar de novo. O corpo é `{ type, aggregateId, correlationId }` , com `type` igual a `EssayCorrected` , `EssayCorrectionFailed` ou `MockExamProcessed` . Nunca carrega texto de redação, nota ou dado pessoal.

Evolução. As rotas são versionadas por prefixo (`/v1`). Mudança compatível (campo opcional novo) não muda a versão; mudança incompatível cria `/v2` e mantém `/v1` até o último app que a usa sair de circulação.

1.8 Custos e recursos gratuitos (D10)

Regra do projeto: toda decisão deve ter opção gratuita, exceto a IA. A tabela abaixo lista cada recurso, o limite gratuito consultado em 30/10/2026 nas páginas oficiais, o que acontece ao passar dele e o custo por 1.000 usuários quando faz sentido. Itens marcados com † vêm de conhecimento anterior e não foram reverificados nesta data.

Recurso	Limite gratuito	Custo ao ultrapassar	Por 1.000 usuários
Oracle Cloud Always Free (D11)	VM Ampere A1 (2 OCPU e 12 GB no total, conforme a página consultada), 2 VMs AMD micro (1 GB cada), 200 GB de disco, 10 TB/mês de saída	Passar a Pay As You Go; o que fica dentro do Always Free segue sem cobrança	US$ 0 (carga a validar em teste)
GitHub Actions e repositório	Uso gratuito para repositório público	Repositório privado tem créditos incluídos limitados; consultar antes de privatizar	US$ 0
GitHub Container Registry †	Gratuito para pacotes públicos	Não verificado	US$ 0

Recurso	Limite gratuito	Custo ao ultrapassar	Por 1.000 usuários
EAS Build (plano Free)	15 builds Android e 15 iOS por mês, fila de baixa prioridade	Starter US$ 19/mês (US$ 45 de crédito de build); Production US$ 199/mês (US$ 225)	US$ 0
EAS Update (opcional)	1.000 usuários ativos por mês	Starter inclui 3.000 e Production 50.000; acima, preço por uso (valor não publicado na página)	US$ 0 até 1.000 usuários ativos
Sentry (plano Developer)	5 mil erros, 1 usuário, 30 dias de histórico	Team: US$ 29/mês (US$ 26 no plano anual)	US$ 0
Grafana Cloud (plano Free)	10 mil séries de métricas, 50 GB de logs, 50 GB de traces, 3 usuários, 14 dias de retenção	Pro a partir de US$ 19/mês mais uso	US$ 0
HTTPS (Let's Encrypt pelo Caddy) †	Gratuito	—	US$ 0
Login com Google e com Apple †	Gratuito (a Apple exige a conta de desenvolvedor, que já existe)	—	US$ 0
Push do Expo	A documentação consultada não informa preço nem limites	Confirmar antes de depender em volume	A confirmar
Credenciais de push do Android (Firebase) †	Projeto Firebase gratuito	—	US$ 0
Domínio e DNS (D13)	Domínio já existente na Hostinger	—	US$ 0
Contas Apple e Google Play (D12)	Já existem; licenças fora do orçamento	—	Fora do escopo
Render (rota de migração)	Redis gratuito de 25 MB sem persistência em disco: incompatível com o BullMQ	Web US$ 7/mês, Postgres US$ 6/mês (1 GB), Key Value US$ 10/mês (256 MB), total US$ 23/mês; Worker separado não verificado	≈ US$ 0,023 por usuário/mês (US$ 23 a cada 1.000)

Custo da IA (GPT 6.1 - Terra), único custo que cresce por usuário. Preços por 1 milhão de tokens na página oficial de preços da OpenAI (modelo `gpt-6.1-terra`, camada padrão): entrada US$ 1,00; entrada em cache US$ 0,05; saída US$ 5,00; gravação em cache de 30 minutos US$ 1,25; de 12 horas US$ 2,50. A mesma página mostra uma segunda linha para o modelo com metade desses valores, que parece ser a camada flex ou batch (o rótulo não está explícito, por isso não é usada nas contas).

As quantidades de tokens abaixo são **estimativas de arquitetura, não medições**: cerca de 3.500 tokens de entrada por correção (texto da redação mais prompt e critérios, dos quais cerca de 1.500 formam um prefixo estável) e 1.000 de saída. A tabela de custo por 1.000 assinantes Premium assume o teto de uso (todos usam as 4 correções do mês, ou seja, 4.000 correções):

Cenário	Custo por correção	Custo por mês a cada 1.000 assinantes Premium
Sem cache (3.500 de entrada e 1.000 de saída)	US$ 0,0085	**US$ 34,00**
Prefixo de 1.500 tokens servido do cache em todas as chamadas	US$ 0,0071	US$ 28,30
Saída de 2.000 tokens, sem cache	US$ 0,0135	US$ 54,00

O custo por assinante fica em no máximo US$ 0,034 por mês no cenário base. Para estimar o custo por mil usuários totais, multiplique pela fração de assinantes: com 10% de Premium, US$ 3,40 a cada 1.000 usuários; com 25%, US$ 8,50. O cache só compensa se o prefixo for reaproveitado: escrever custa 25% a mais que a entrada comum e cada leitura economiza cerca de 95%, então basta uma leitura a cada quatro escritas para valer a pena. Todos os números serão substituídos pelo consumo real registrado pelo Worker (tabela `ai_usage`, guia do Back-End) logo nas primeiras semanas.

Custos que dependem do Premium e ainda não foram decididos. Se o Premium for vendido pelas lojas, as comissões consultadas hoje são: Apple 15% no Small Business Program (receita de até US$ 1 milhão no ano anterior); Google Play 15% sobre assinaturas nos mercados que não estão entre os de estrutura nova (a página lista Austrália, Espaço Econômico Europeu, Japão, Reino Unido e EUA com 10% mais 5% de cobrança). Se for usado o RevenueCat para gerir assinaturas, a página informa que não há cobrança até US$ 2.500 de receita mensal rastreada e 1% acima disso (a página não esclarece se o 1% incide só sobre o excedente). Se o Premium precisa ou não usar a cobrança das lojas depende das diretrizes de cada loja e deve ser confirmado antes de implementar. Nada disso está aprovado: o item segue em aberto (§14 do documento de arquitetura).

1.9 Glossário

Termo	Significado
Comando	Intenção do aluno de mudar o estado; verbo no imperativo (`SubmitEssay`). Responde `202`, nunca o resultado.
Consulta	Leitura de um modelo de leitura; nunca muda estado; devolve `asOf`.
Evento	Fato já ocorrido e imutável, nomeado no passado (`EssaySubmitted`).
`commandId`	UUID gerado no app; chave de idempotência e `correlationId` de toda a cadeia.
Outbox local	Fila de comandos pendentes no aparelho, enviada quando houver conexão.
Outbox transacional	Tabela do servidor gravada na mesma transação do estado; um relay publica na fila.
Relay	Processo que lê o outbox transacional e publica no BullMQ.
Worker	Processo sem servidor HTTP que consome uma fila (correção de redação, simulados, projeções).
Projeção (read model)	Tabela desnormalizada atualizada por eventos, usada só por consultas.
`asOf`	Instante da última projeção aplicada; indica o possível atraso do dado.
Dead-letter	Fila de mensagens que esgotaram as tentativas ou falharam de forma permanente.
AOF	Persistência do Redis por log de comandos, exigida pelo BullMQ em produção.
Premium	Plano com direito à correção de redação por IA (4 por mês).

Termo	Significado
MAU	Usuários ativos por mês (base de cobrança do EAS Update).

1.10 Fontes consultadas em 30/10/2026

Preços, limites e versões deste guia vêm das páginas oficiais abaixo. Quem for implementar deve reconferi-las, porque planos e limites mudam.

- Oracle Always Free: `docs.oracle.com/en-us/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm`.
- BullMQ: `docs.bullmq.io/guide/going-to-production`, `.../jobs/job-ids` e `.../retrying-failing-jobs`; tipos do pacote `bullmq` 6.5.2.
- Expo: `expo.dev/pricing` e `docs.expo.dev/push-notifications/overview`.
- Render: `render.com/pricing`. Sentry: `sentry.io/pricing`. Grafana: `grafana.com/pricing`.
- OpenAI: `developers.openai.com/api/docs/pricing` (linha `gpt-6.1-terra`).
- Lojas e assinaturas: `developer.apple.com/app-store/small-business-program`, `support.google.com/googleplay/android-developer/answer/112622` e `revenuecat.com/pricing`.
- Ciclo de vida: `nodejs.org/en/about/previous-releases`, `endoflife.date/postgresql` e `endoflife.date/redis`.
- Versões e licenças de pacotes: registro do npm, consultado em 30/10/2026.

2. Ponto de partida e responsabilidades do DevOps

Não existe infraestrutura, pipeline nem contêiner. O repositório não tem `.github/`, `infra/`, `eas.json` nem `Dockerfile`. As duas últimas pastas nascem no primeiro uso (§10.2 da arquitetura). O que existe: um repositório **público**, as contas de desenvolvedor da Apple e do Google Play (D12) e um domínio na Hostinger (D13).

As responsabilidades são sete:

1. Provisionar e manter a VM da Oracle (D11) e tudo o que roda nela.
2. Garantir os requisitos de Redis do BullMQ: persistência AOF e política `noeviction`.
3. Entregar o backend com segurança e com possibilidade de retorno (CI/CD).
4. Entregar o app (builds EAS e lojas) dentro dos limites gratuitos.
5. Observar o sistema e alertar sobre o que importa: `dead-letter`, atraso de filas, outbox parado, custo da IA.
6. Fazer backup e provar que a restauração funciona.
7. Manter os custos dentro dos limites gratuitos e avisar antes de qualquer cobrança (D10).

Regra especial por causa do repositório público. Tudo o que for versionado é visível: arquivos de `infra/`, fluxos de CI e seus logs. Segredos, chaves, IPs privados e dados de alunos nunca entram no repositório nem aparecem em logs de CI. Pull requests vindos de forks não recebem segredos; use o evento `pull_request` (e não `pull_request_target`) nos fluxos de teste.

3. Topologia de produção e dimensionamento

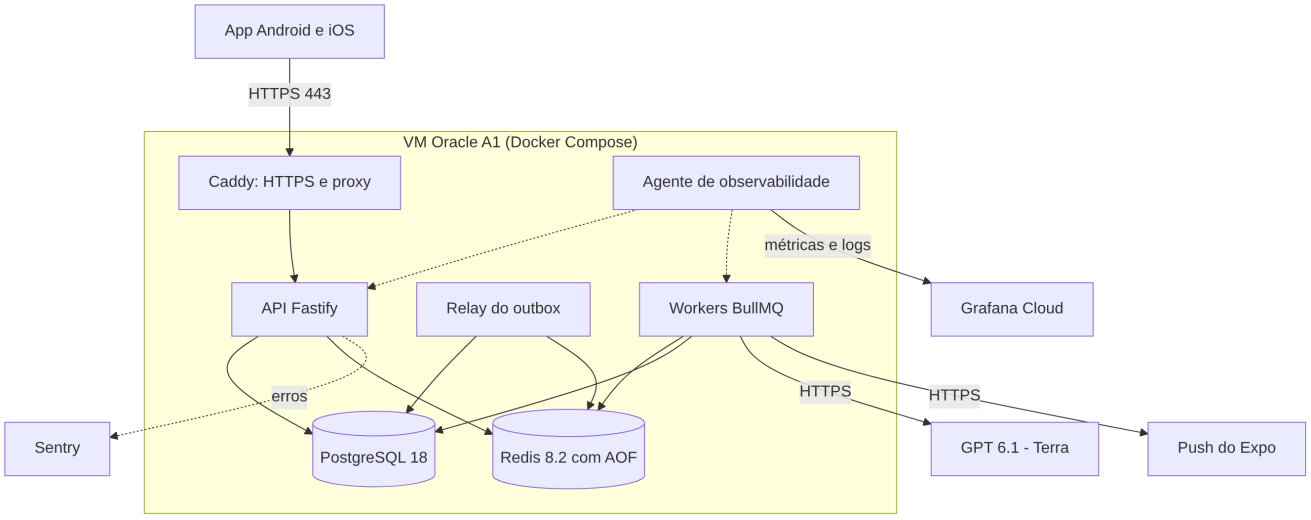


Figura 1 — Topologia de produção

3.1 Princípios

- **Uma porta pública.** Somente o Caddy publica 80 e 443. API, Workers, relay, PostgreSQL e Redis não publicam portas no host.
- **Dois níveis de rede.** PostgreSQL e Redis ficam em uma rede interna do Compose (`internal: true`), sem acesso à internet. A API fala com o Caddy. Workers e relay também têm saída para a internet (IA e push).
- **Processos separados.** API, Workers e relay usam a mesma imagem com comandos diferentes e podem ser reiniciados e escalados independentemente (§7 da arquitetura).
- **Tudo reprodutível por script.** Nenhuma configuração existe só na VM: Compose, Caddyfile, unidades do systemd e scripts vivem em `infra/`, e a VM pode ser recriada do zero (runbook R6).

3.2 Limites do plano gratuito da Oracle

Conforme a página oficial consultada em 30/10/2026: Ampere A1 com **2 OCPUs e 12 GB** de memória no total (1.500 horas de OCPU e 9.000 horas de GB por mês), até duas instâncias A1 ou uma com todos os recursos; até **duas VMs AMD micro** (`VM.Standard.E2.1.Micro`, 1/8 de OCPU e 1 GB cada, 50 Mbps); **200 GB** de disco somando boot e blocos, mais 5 backups de volume; **10 TB** de saída por mês; recursos apenas na **região de origem** da conta. O tamanho do volume de boot padrão (50 GB) difere do mínimo citado (47 GB) na própria página: confirme no console. Os números mudam; reconfira antes de provisionar.

3.3 Orçamento de memória e disco (estimativa a validar no marco B7)

Componente	Memória prevista	Observação
PostgreSQL	até 3 GB	`shared_buffers` 1 GB e conexões
Redis	até 1 GB	`maxmemory` de 512 MB (jobs são pequenos: só identificadores)
API	512 MB	
Workers	1 GB	`concurrency` pequena; o trabalho pesado é a chamada de rede à IA

Componente	Memória prevista	Observação
Relay	256 MB	
Caddy	128 MB	
Agente, exportadores e Docker	768 MB	
Sistema operacional	512 MB	
Total previsto	**≈ 7,2 GB de 12 GB**	Margem para picos

Crescimento de dados (estimativa): 1.000 assinantes Premium usando as 4 correções geram 4.000 redações por mês; com texto e resultado na ordem de 5 a 15 KB cada, são cerca de 20 a 60 MB por mês. O disco não é o gargalo; o que importa é o backup e a retenção. Todos esses valores são premissas e serão medidos.

4. Provisionamento na Oracle Cloud

4.1 Antes de começar

- **A região de origem é definida ao criar a conta e não muda.** Os recursos Always Free só podem ser criados nela. Escolha a de menor latência para o Brasil (São Paulo ou Vinhedo †) e confirme que ela oferece Ampere A1.
- A conta pode exigir um cartão para verificação de identidade †. A página oficial diz que, ao migrar para Pay As You Go, a Oracle não cobra o que fica dentro dos limites Always Free e cobra o uso acima deles. Qualquer upgrade é uma decisão que precisa de aprovação explícita (D10).

4.2 Passos no console

1. **Rede.** Criar uma VCN com sub-rede pública (o assistente “VCN com acesso à internet”). Em grupos de segurança de rede (NSG) ou na lista de segurança: entrada `80` e `443` de `0.0.0.0/0`; entrada `22` **somente dos IPs da equipe**; saída liberada.
2. **Instância.** Forma `VM.Standard.A1.Flex` (arquitetura ARM64) com o sistema Ubuntu LTS para ARM. Dimensione pelo orçamento do §3.3 e **não precisa usar os 12 GB**: veja o risco de ociosidade abaixo. Volume de boot padrão. Chave SSH `ed25519` da equipe (a chave privada nunca sai da máquina de quem a gerou).
3. **IP.** Anotar o IP público e confirmar no console se o IP reservado é gratuito enquanto anexado † (um IP efêmero muda se a instância for recriada, e o DNS do §8 aponta para ele).
4. **Capacidade.** Se aparecer “Out of host capacity”, a página oficial indica tentar outro domínio de disponibilidade (A1 pode ficar em qualquer um, exceto na Coreia do Sul Norte), esperar e tentar de novo, ou migrar para Pay As You Go (decisão de custo).

4.3 Risco: recuperação de instância ociosa

A Oracle pode recuperar instâncias Always Free que considera ociosas. Pelo critério publicado, é ociosa a instância que, em 7 dias, tem simultaneamente: percentil 95 de CPU abaixo de 20%, rede abaixo de 20% e (somente em A1) memória abaixo de 20%.

- **Dimensionar pelo que se usa.** A memória das instâncias A1 conta. Uma VM de 12 GB com 3 GB em uso fica em 25% e, com carga baixa, perto do limite; dimensionar a VM para o consumo real (e

não o máximo gratuito) é legítimo e também reduz o risco.

- **Nada de carga artificial.** Gerar tráfego falso para escapar da regra é contornar a política do provedor e não é feito.
- **Plano de recuperação.** O risco é aceito porque o runbook R6 recria a VM por script e restaura o último backup em cerca de duas horas (proposta de RTO). O alerta do §13 acompanha CPU, memória e rede para antecipar o cenário.
- **A confirmar com a Oracle**: se a conta Pay As You Go é isenta da recuperação por ociosidade. A página consultada não diz.

5. Preparação e endurecimento do servidor

Todo o procedimento vira o script `infra/scripts/bootstrap-host.sh` (idempotente), para que o runbook R6 seja repetível. Itens:

- **Usuários.** Um usuário de operação com `sudo` e um usuário `deploy` sem `sudo`, dono de `/opt/aprova`. Login de `root` por SSH desabilitado.
- **SSH.** `PasswordAuthentication no`, `PermitRootLogin no`, somente chaves. A porta 22 só aceita os IPs da equipe na rede da Oracle (§4.2).
- **Firewall do host.** As imagens Ubuntu da Oracle costumam trazer regras de `iptables` que bloqueiam 80 e 443 mesmo com a regra de rede aberta no console †. Verifique com `sudo iptables -L INPUT -n --line-numbers` e ajuste de forma persistente (`netfilter-persistent save`). Atenção: portas publicadas pelo Docker contornam o `ufw`; por isso **só o Caddy** publica portas.
- **Atualizações.** `unattended-upgrades` para correções de segurança; janela mensal para atualizações maiores, com a imagem dos contêineres fixada.
- **fail2ban** para SSH.
- **Hora.** Sincronização de relógio ativa (os tokens e a cota dependem de hora correta).
- **Memória virtual.** 2 GB de swap como rede de segurança.
- **Ajustes para o Redis.** `vm.overcommit_memory = 1` e página enorme transparente desabilitada, que são as duas recomendações que o próprio Redis emite no log de inicialização.
- **Docker.** Instalado do repositório oficial; `daemon.json` com rotação de logs (`json-file`, `max-size` de 10 MB, `max-file` de 5) e `live-restore`.
- **Diretórios.** `/opt/aprova` (repositório de `infra/`, `compose.yml`), `/opt/aprova/secrets` (modo 700), `/opt/aprova/backups`.

6. Docker Compose de produção

6.1 Estrutura de `infra/`

```
infra/
├── compose.yml             # produção
├── compose.dev.yml         # desenvolvimento local (Postgres e Redis)
├── Caddyfile
├── scripts/                # bootstrap-host.sh, deploy.sh, backup.sh, restore-drill.sh
├── systemd/                # aprova-deploy.{service,timer}, aprova-backup.{service,timer}
└── README.md
```

O `Dockerfile` do backend fica em `backend/Dockerfile`, e o contexto de build é `backend/`.

6.2 `backend/Dockerfile`

O backend não tem etapa de build: o Node 24 executa o TypeScript diretamente e `contracts/` só contém tipos, então a imagem leva apenas `src`, `migrations` e as dependências de produção. Todas as dependências propostas são JavaScript puro, o que simplifica a imagem para ARM64.

```
FROM node:24-bookworm-slim AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

FROM node:24-bookworm-slim
ENV NODE_ENV=production
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY package.json ./
COPY src ./src
COPY migrations ./migrations
USER node
CMD ["node", "src/bootstrap/api.ts"]
```

Fixe as imagens por versão menor (e, quando possível, por digest) depois de confirmar as tags no Docker Hub; os exemplos acima e abaixo usam tags de referência. As imagens oficiais do Node, PostgreSQL, Redis e Caddy são multiarquitetura (confirme com `docker manifest inspect`).

6.3 `infra/compose.yml`

```
name: aprovaai

x-app: &app
  image: ghcr.io/gwbr-technologies/aprovaai-backend:${IMAGE_TAG:-production}
  restart: unless-stopped
  logging: { driver: json-file, options: { max-size: 10m, max-file: "5" } }

services:
  caddy:
    image: caddy:2
    restart: unless-stopped
    ports: ["80:80", "443:443", "443:443/udp"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    depends_on: { api: { condition: service_healthy } }
    networks: [web]

  api:
    <<: *app
    command: ["node", "src/bootstrap/api.ts"]
    env_file: [/opt/aprova/secrets/common.env, /opt/aprova/secrets/api.env]
    deploy: { resources: { limits: { memory: 512M } } }
    healthcheck:
```

```
      test: ["CMD", "node", "-e", "fetch('http://127.0.0.1:3000/readyz').then(r=>process.exit(r.ok?
0:1)).catch(()=>process.exit(1))"]
      interval: 15s
      timeout: 5s
      retries: 5
      start_period: 20s
    depends_on:
      migrate: { condition: service_completed_successfully }
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    networks: [web, data]

  worker:
    <<: *app
    command: ["node", "src/bootstrap/worker.ts"]
    env_file: [/opt/aprova/secrets/common.env, /opt/aprova/secrets/worker.env]
    stop_grace_period: 150s
    deploy: { resources: { limits: { memory: 1G } } }
    depends_on:
      migrate: { condition: service_completed_successfully }
      redis: { condition: service_healthy }
    networks: [data, egress]

  relay:
    <<: *app
    command: ["node", "src/bootstrap/relay.ts"]
    env_file: [/opt/aprova/secrets/common.env, /opt/aprova/secrets/relay.env]
    deploy: { resources: { limits: { memory: 256M } } }
    depends_on:
      migrate: { condition: service_completed_successfully }
      redis: { condition: service_healthy }
    networks: [data]

  migrate:
    <<: *app
    restart: "no"
    command: ["node", "src/bootstrap/migrate.ts"]
    env_file: [/opt/aprova/secrets/common.env, /opt/aprova/secrets/migrate.env]
    depends_on: { postgres: { condition: service_healthy } }
    networks: [data]

  postgres:
    image: postgres:18
    restart: unless-stopped
    environment:
      POSTGRES_DB: aprovaai
      POSTGRES_USER: aprova_admin
      POSTGRES_PASSWORD_FILE: /run/secrets/postgres_password
    secrets: [postgres_password]
    volumes: [pg_data:/var/lib/postgresql]
    command: ["postgres", "-c", "shared_buffers=1GB", "-c", "effective_cache_size=4GB",
              "-c", "max_connections=100", "-c", "work_mem=16MB"]
```

```
    deploy: { resources: { limits: { memory: 3G } } }
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U aprova_admin -d aprovaai"]
      interval: 10s
      timeout: 5s
      retries: 10
    networks: [data]

  redis:
    image: redis:8.2
    restart: unless-stopped
    secrets: [redis_password]
    command: ["sh", "-c", "exec redis-server --appendonly yes --appendfsync everysec --maxmemory 512mb --
maxmemory-policy noeviction --requirepass \"$$(cat /run/secrets/redis_password)\""]
    volumes: [redis_data:/data]
    deploy: { resources: { limits: { memory: 1G } } }
    healthcheck:
      test: ["CMD-SHELL", "redis-cli --no-auth-warning -a \"$$(cat /run/secrets/redis_password)\" ping | grep
-q PONG"]
      interval: 10s
      timeout: 5s
      retries: 10
    networks: [data]

networks:
  web: {}
  data: { internal: true }
  egress: {}

volumes:
  caddy_data: {}
  caddy_config: {}
  pg_data: {}
  redis_data: {}

secrets:
  postgres_password: { file: /opt/aprova/secrets/postgres_password }
  redis_password: { file: /opt/aprova/secrets/redis_password }
```

Pontos que a equipe precisa entender, linha a linha:

- `x-app` evita repetir a imagem e a rotação de logs. Cada serviço lê `common.env` mais o seu próprio arquivo de ambiente, o que dá a cada processo só os segredos de que precisa: `OPENAI_API_KEY` existe apenas em `worker.env`, e cada serviço usa um papel de banco diferente (`api.env`, `worker.env` e `relay.env` com papéis restritos; `migrate.env` com o papel que cria o esquema e os papéis, guia do Back-End §16). A etiqueta `production` é a que o servidor persegue (§11); `IMAGE_TAG` fixa um commit específico no rollback.
- `migrate` roda antes de API, Workers e relay (`service_completed_successfully`), com o mesmo código. Migrações precisam ser compatíveis com a versão anterior (primeiro expandir, depois contrair), porque as migrações **não são revertidas automaticamente**.

- `postgres` e `redis` só estão na rede `data` (`internal: true`), sem saída para a internet e sem portas publicadas.
- `worker` tem `stop_grace_period` de 150 s, maior que o tempo limite de um job da IA, para o `worker.close()` terminar o job em andamento (§9.3 do guia do Back-End). O valor deve acompanhar `AI_TIMEOUT_MS`.
- `redis` lê a senha de um arquivo de segredo (sem aparecer em `docker inspect`) e aplica os requisitos do BullMQ: `appendonly yes` com `appendfsync everysec` e `maxmemory-policy noeviction`.
- Os limites de memória (`deploy.resources.limits`) refletem o orçamento do §3.3 e são pontos de partida. O `postgres` fixa `shared_buffers` e afins; revise depois do teste de carga.
- A imagem do PostgreSQL 18 mudou o layout do diretório de dados; o volume é montado em `/var/lib/postgresql` †. Confira a documentação da imagem antes do primeiro uso.

7. Redis e PostgreSQL: configuração exigida

7.1 Redis (exigências do BullMQ)

Fonte: guia “Going to production” do BullMQ, consultado em 30/10/2026.

Exigência	Configuração
Nada de remoção de chaves	`maxmemory-policy noeviction`
Persistência	AOF (`appendonly yes`), com sincronização por segundo (`appendfsync everysec`), que a documentação considera suficiente para a maioria das aplicações
Instância dedicada	O Redis serve só o BullMQ
Memória limitada	`maxmemory 512mb` (proposta); com `noeviction`, ao encher, as escritas falham: alerta em 70%
Senha	`requirepass`; sem porta publicada

As opções de conexão do cliente (`maxRetriesPerRequest: null` nos Workers e `enableOfflineQueue: false` nos produtores) estão no guia do Back-End (§9.1).

7.2 PostgreSQL

- Dois esquemas (`write` e `read`) numa instância; papéis de menor privilégio (guia do Back-End, §16).
- Fusos e relógio em UTC; a regra de mês de `America/Sao_Paulo` é aplicada na aplicação.
- Extensões: nenhuma obrigatória.
- Monitorar conexões, tamanho do banco e idade do último backup (§13 e §14).

8. DNS, domínio e HTTPS

O domínio já existe na Hostinger (D13). **A API usa só um subdomínio** (`api.<seu-domínio>`), então o site e os e-mails do domínio principal ficam intactos.

8.1 DNS

1. No painel da Hostinger, abra a zona DNS do domínio. Se o domínio usa servidores de nomes externos, crie o registro lá.

2. Crie um registro `A` com nome `api` apontando para o IP público da VM, com TTL curto (300 s) durante a implantação; depois de estável, 3600 s. O plano de hospedagem Business da Hostinger não participa: a VM da Oracle serve a API e o plano continua servindo o que já serve.
3. Verifique a propagação com `dig +short api.<seu-domínio>`, antes de subir o Caddy (a emissão do certificado depende do DNS correto e das portas 80 e 443 acessíveis).

8.2 HTTPS automático com Caddy

O Caddy obtém e renova certificados da Let's Encrypt † sozinho. `infra/Caddyfile`:

```
{
    email ops@<seu-domínio>
}

api.<seu-domínio> {
    encode zstd gzip
    request_body {
        max_size 512KB
    }
    header {
        Strict-Transport-Security "max-age=31536000"
        -Server
    }
    reverse_proxy api:3000 {
        health_uri /healthz
    }
    log {
        output stdout
        format json
    }
}
```

- **Teste primeiro no ambiente de testes da Let's Encrypt** (`acme_ca https://acme-staging-v02.api.letsencrypt.org/directory` nas opções globais): emissões que falham repetidamente esbarram em limites de taxa †. Remova a linha depois de validar.
- `/metrics` **não** é roteado: as métricas ficam numa porta interna e nunca saem pelo proxy.
- Ajuste `max_size` ao limite de texto da redação e ao `bodyLimit` da API (ambos definidos com o produto).
- A política de nomes e o `email` de contato são do responsável pelo domínio.

9. Segredos e configuração

Segredo	Onde fica	Quem usa
Senhas do PostgreSQL e do Redis	`/opt/aprova/secrets/postgres_password` e `redis_password` (modo 600)	Compose (secrets)
`common.env` (`REDIS_URL`, `JWT_SECRET`, `JWT_ISSUER`, `JWT_AUDIENCE`, `SENTRY_DSN`, limites e demais variáveis não sensíveis)	`/opt/aprova/secrets/common.env` (modo 600)	Todos os serviços

Segredo	Onde fica	Quem usa
`api.env`, `worker.env`, `relay.env`, `migrate.env` (cada um com a sua `DATABASE_URL` de papel restrito; `OPENAI_API_KEY` só em `worker.env`)	`/opt/aprova/secrets/` (modo 600)	O serviço correspondente
Chave privada de backup (`age`)	Fora da VM, com a equipe	Restauração
Token do EAS (`EXPO_TOKEN`), se o CI móvel for usado	Segredos do GitHub	Fluxo de build
Credenciais de loja (chave da App Store Connect, conta de serviço do Google Play)	Gerenciadas pelo EAS	`eas submit`

Regras:

- **Nada disso entra no repositório**, que é público. Antes do primeiro uso, `.env` entra no `.gitignore` (hoje só `.env*.local` está coberto).
- Geração: `openssl rand -base64 48` para `JWT_SECRET` e para as senhas.
- **Rotação.** `JWT_SECRET`: troque o arquivo e reinicie a API; os tokens de acesso (15 min) deixam de valer e os apps renovam com o refresh token. Chave da IA: crie a nova, atualize o arquivo, `docker compose up -d worker`, revogue a antiga. Senhas de banco: janela de manutenção.
- **Vazamento** de qualquer segredo: runbook R10.
- A lista de variáveis e o que é segredo está no guia do Back-End (§18). O SDK de monitoramento recebe o DSN do Sentry, que não é segredo mas não deve ficar em código.
- O varredor de segredos do GitHub e uma verificação com `gitleaks` no CI (opcional) reduzem o risco de commit acidental.

10. Integração contínua

GitHub Actions é gratuito para repositório público (§1.8). Se o repositório for tornado privado, revise a cota antes. Fixe cada action por SHA de commit (e atualize por Dependabot): as versões abaixo são as de referência da escrita e precisam ser conferidas.

10.1 `.github/workflows/ci.yml`

```
name: ci
on:
  pull_request:
  push:
    branches: [development, master]

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  app:
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test --if-present

  backend:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: backend } }
    services:
      postgres:
        image: postgres:18
        env: { POSTGRES_PASSWORD: ci-only, POSTGRES_DB: aprovaai_test }
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres" --health-interval 5s
          --health-timeout 5s --health-retries 10
      redis:
        image: redis:8.2
        ports: ["6379:6379"]
        options: >-
          --health-cmd "redis-cli ping" --health-interval 5s
          --health-timeout 5s --health-retries 10
    env:
      DATABASE_URL: postgres://postgres:ci-only@localhost:5432/aprovaai_test
      REDIS_URL: redis://localhost:6379
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 24
          cache: npm
          cache-dependency-path: backend/package-lock.json
      - run: npm ci
      - run: npm run typecheck
      - run: npm run lint
      - run: npm test
```

Notas:

- O fluxo só funciona depois que os marcos F0 (scripts do app) e B0 (scripts do backend) existirem; antes disso o job `app` reprovaria por falta de script. A senha `ci-only` é de um contêiner descartável de teste e não é segredo.
- Os testes de integração do BullMQ rodam contra o Redis do serviço, e os de persistência contra o PostgreSQL do serviço.
- `permissions: contents: read` e `concurrency` reduzem a superfície e o desperdício.
- O app roda no Node 22 (versão usada para validar o bundle do Android neste trabalho) e o backend no Node 24; ambas são linhas LTS (`nodejs.org`, 30/10/2026).

10.2 Proteção de ramos e hábitos

- `master` e `development` protegidos: pull request obrigatório, verificações `app` e `backend` obrigatórias, sem push forçado.
- Dependabot para `npm` (raiz e `backend/`), `github-actions` e `docker`, em `.github/dependabot.yml`.
- Varredura de imagem com Trivy no fluxo de entrega (gratuito) e `npm audit` registrados.
- Comandos de qualidade documentados no `README.md`. Quando o fluxo estiver ativo, `npm run lint` passa a ser a verificação oficial do projeto.

11. Entrega contínua do backend

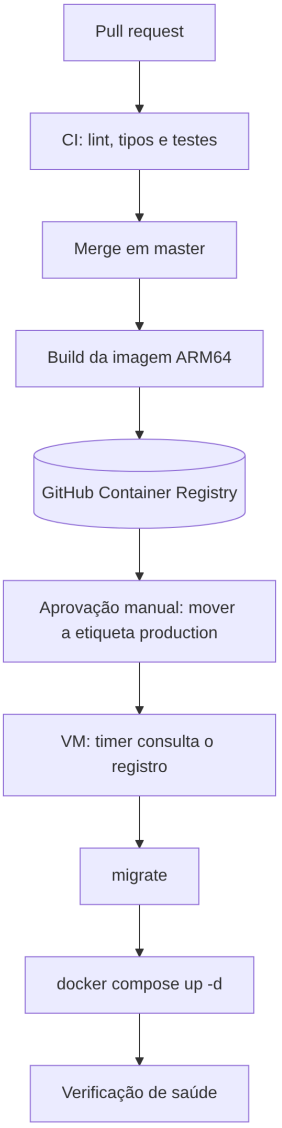


Figura 2 — Entrega contínua do backend

Escolha: entrega por consulta (pull), não por SSH. Deploy por SSH a partir do GitHub exigiria deixar a porta 22 aberta para os IPs dos executores do GitHub, que mudam. Na entrega por consulta, a VM verifica o registro periodicamente e nunca recebe conexão de entrada do CI: a porta 22 continua restrita à equipe.

11.1 Publicação da imagem

No merge em `master`, um fluxo constrói a imagem para `linux/arm64`, publica em `ghcr.io/gwbr-technologies/aprovaai-backend:<sha>` e **não** mexe em produção. Por ser JavaScript puro, o build ARM64 por emulação (QEMU) é rápido. O repositório é público, então o pacote também pode ser público e a VM não precisa de credencial para baixá-lo; a imagem contém apenas código que já está no repositório, nunca segredos (confirme a visibilidade e a gratuidade do pacote nas configurações do GitHub †).

```
# .github/workflows/backend-image.yml (esqueleto)
name: backend-image
on:
  push:
    branches: [master]
    paths: ["backend/**"]
permissions:
  contents: read
  packages: write
jobs:
  image:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-qemu-action@v3
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: backend
          platforms: linux/arm64
          push: true
          tags: ghcr.io/gwbr-technologies/aprovaai-backend:${{ github.sha }}
```

11.2 Promoção para produção

Um segundo fluxo, manual (`workflow_dispatch`) e protegido por um ambiente `production` com aprovação obrigatória, move a etiqueta `production` para o `<sha>` escolhido:

```
docker buildx imagetools create \
  -t ghcr.io/gwbr-technologies/aprovaai-backend:production \
  ghcr.io/gwbr-technologies/aprovaai-backend:<sha>
```

11.3 O que a VM faz

Um temporizador do systemd (`aprova-deploy.timer`, a cada 2 minutos) executa `infra/scripts/deploy.sh`. O Compose só recria os contêineres cuja imagem mudou:

```
#!/usr/bin/env bash
set -euo pipefail
```

```
cd /opt/aprova
docker compose pull --quiet
docker compose up -d --remove-orphans
docker image prune -f
```

O serviço `migrate` do Compose já roda antes de API, Workers e relay (§6.3), de modo que a migração acompanha cada versão nova. Depois do `up`, a verificação de saúde do Compose (`/readyz`) decide se a API está de pé; o fluxo de promoção também consulta `https://api.<seu-domínio>/readyz` e falha se a versão nova não responder.

11.4 Rollback

Rollback de **código**: rode o fluxo de promoção apontando para o `<sha>` anterior; a VM volta em até 2 minutos. Em emergência, na VM: `IMAGE_TAG=<sha> docker compose up -d`.

Rollback de **dados**: migrações não se revertem sozinhas. Por isso toda migração é compatível com a versão anterior (expandir e contrair em dois lançamentos). Se uma migração corromper dados, o caminho é restaurar o backup (runbook R3) e republicar as mensagens pendentes (§8.4 do guia do Back-End).

11.5 Regras de entrega

- Nada vai para produção sem `app` e `backend` verdes e sem aprovação manual.
- Uma versão por vez; a etiqueta `production` sempre aponta para um `<sha>` conhecido, e o `<sha>` anterior é anotado no relatório do deploy.
- Cada deploy registra: o `<sha>`, o horário, quem aprovou e o resultado de `/readyz`.

12. Pipeline móvel e lojas

12.1 EAS Build (plano gratuito)

O plano Free tem 15 builds Android e 15 iOS por mês, em fila de baixa prioridade (§1.8). Para não esgotar a cota:

- Não construir a cada push. Construir **manualmente** ou ao criar uma etiqueta de versão (`v*`).
- Build de desenvolvimento (`developmentClient`) só quando uma dependência nativa muda.
- Build `preview` para candidatos a versão e `production` somente para enviar às lojas.
- Builds locais (`eas build --local`) usam a máquina da equipe e não consomem a cota †.
- Quando a cota apertar, a equipe é avisada **antes**: o plano Starter custa US$ 19 por mês (US$ 45 de crédito de build).

`eas.json` com o ambiente de cada perfil (a URL da API é pública):

```
{
  "cli": { "appVersionSource": "remote" },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "env": { "EXPO_PUBLIC_API_URL": "https://api.<seu-domínio>" }
    },
    "preview": {
```

```
      "distribution": "internal",
      "env": { "EXPO_PUBLIC_API_URL": "https://api.<seu-domínio>" }
    },
    "production": {
      "autoIncrement": true,
      "env": { "EXPO_PUBLIC_API_URL": "https://api.<seu-domínio>" }
    }
  },
  "submit": { "production": {} }
}
```

12.2 Credenciais

- **Android:** o EAS gera e guarda a chave de assinatura. Para o push, é preciso um projeto no Firebase (gratuito †) e a configuração do FCM no EAS. Guarde o arquivo `google-services.json` fora do repositório se ele contiver chaves que não devem ser públicas; confirme na documentação o que pode ser versionado.
- **iOS:** certificados e perfis de provisionamento ficam no EAS, na conta Apple existente. A chave de APNs para push também é gerenciada pelo EAS.
- Chave da App Store Connect e conta de serviço do Google Play, para `eas submit`: carregadas no EAS, nunca no repositório.

12.3 Lojas

As contas existem (D12). Pontos que dependem do produto e bloqueiam a publicação: `bundleIdentifier` e `package` (permanentes), política de privacidade em URL pública (pode ficar no domínio, em `https://<seu-domínio>/privacidade`, fora desta documentação), formulário de segurança de dados e rótulos de privacidade (incluindo envio do texto da redação ao provedor de IA), conta de teste para a revisão e os requisitos de teste fechado do Google Play para contas novas †. Use TestFlight e teste interno do Google Play antes da produção.

12.4 Atualizações OTA (opcional)

EAS Update é gratuito até 1.000 usuários ativos por mês (Starter inclui 3.000; Production, 50.000). Sem EAS Update, toda mudança segue por versão de loja e não há custo. Se for adotado, a contagem de usuários ativos precisa ser acompanhada, e a equipe avisa antes de ultrapassar o limite.

13. Observabilidade e alertas

13.1 Ferramentas gratuitas (P9)

Ferramenta	Plano gratuito (30/10/2026)	Uso
Sentry	5 mil erros, 1 usuário, 30 dias	Erros do app e do backend
Grafana Cloud	10 mil séries de métricas, 50 GB de logs, 50 GB de traces, 3 usuários, 14 dias de retenção; 100 mil execuções de testes de API por mês (Synthetics)	Métricas, logs, painéis, alertas e verificação externa de `/readyz`

Limites que exigem disciplina: **nunca use `userId`, `commandId` ou `essayId` como rótulo de métrica** (cardinalidade explode e ultrapassa as 10 mil séries); logs são de volume controlado (sem texto de

redação, §15 do guia do Back-End); o Sentry usa amostragem e filtro de ruído para não esgotar os 5 mil erros.

13.2 Coleta

Um contêiner do **Grafana Alloy** na VM raspa as métricas internas da API e dos Workers (porta `METRICS_PORT`, só na rede do Compose), lê os logs dos contêineres e os envia ao Grafana Cloud. Os exportadores de host, Redis e PostgreSQL entram no mesmo Compose, na rede interna. A configuração do agente é gerada pelo assistente de integração do próprio Grafana Cloud e versionada em `infra/` **sem tokens** (o token fica em `/opt/aprova/secrets`).

13.3 Painéis mínimos

API (taxa, erros por rota e latência), filas (profundidade, idade do job mais antigo, falhas e crescimento da `dead-letter`, por fila), outbox (pendentes e idade da mais antiga), IA (chamadas por resultado, tokens, custo acumulado, rejeições por Premium e por cota), Redis (memória e persistência), PostgreSQL (conexões, tamanho, idade do último backup) e host (CPU, memória, disco, rede).

13.4 Alertas (limiares são propostas a calibrar no B7)

Alerta	Condição	Gravidade
`dead-letter` cresceu	Qualquer mensagem nova	**Alta**: notificar a equipe imediatamente
API fora do ar	`/readyz` falha por 2 minutos (verificação externa)	Alta
Erros da API	Respostas `5xx` acima de 2% por 5 minutos	Alta
Latência dos comandos	`p95` acima de 1 s por 10 minutos	Média
Outbox parado	Linha pendente mais antiga com mais de 2 minutos	Alta
Correção de redação atrasada	Job mais antigo da fila `essay-correction` com mais de 10 minutos	Média
Projeção atrasada	Job mais antigo da fila `projections` com mais de 2 minutos	Média
Falha da IA	Mais de 20% de falhas em 10 minutos	Alta
Custo da IA	80% do orçamento mensal (`AI_MONTHLY_BUDGET_USD`) ou gasto diário acima do esperado	Média
Redis cheio	Memória acima de 70% de `maxmemory`	Alta (com `noeviction`, escritas passam a falhar)
PostgreSQL	Conexões acima de 80 ou disco acima de 80%	Média
Backup velho	Último backup com mais de 26 horas	Alta
Host	Disco acima de 80% ou memória acima de 85%	Média
Risco de ociosidade (informativo)	CPU, memória e rede abaixo de 20% por 5 dias	Baixa (§4.3)
Certificado	Validade abaixo de 14 dias (verificação externa)	Média

Notificações por e-mail no plano gratuito. Todo alerta aponta para o runbook correspondente (§18).

14. Backups e recuperação

14.1 PostgreSQL

- **Diário**, às 03:30 no horário de Brasília, por temporizador do systemd: `pg_dump` em formato personalizado (`-Fc`) de todo o banco, comprimido, **criptografado com** `age` (a chave pública fica na VM, a privada fica fora dela, com a equipe).
- **Retenção:** 7 diários, 4 semanais e 3 mensais.
- **Destino:** disco local (`/opt/aprova/backups`) mais uma cópia **fora da VM**. O destino gratuito de cópia externa a confirmar: o armazenamento de objetos da Oracle (verificar no console se há cota Always Free e qual) ou outro armazenamento que a empresa já pague. Uma cópia só na VM não é backup (a VM pode ser recuperada pelo provedor, §4.3).
- **Meta proposta:** perda máxima de 24 horas (RPO) e retorno em até 2 horas (RTO). Para RPO menor, arquivar o WAL (por exemplo com `pgBackRest` ou `WAL-G`, ambos de código aberto) é uma evolução futura.

14.2 Redis

O Redis não é fonte de verdade. Perder o Redis significa perder jobs em fila, não dados: o **reconciliador** (§8.4 do guia do Back-End) republica redações e simulados que ficaram presos, e um script reconstrói projeções a partir de `write.*`. O AOF existe para reduzir esse caso, não para substituí-lo. Os arquivos AOF podem entrar no backup local por conveniência.

14.3 Simulado de restauração

Todo mês, `infra/scripts/restore-drill.sh` restaura o último backup num contêiner descartável, verifica contagens mínimas e o resultado de uma consulta de sanidade e **registra a duração**. Backup sem restauração testada não conta.

15. Ambientes: desenvolvimento local e staging

15.1 Desenvolvimento local

`infra/compose.dev.yml` sobe PostgreSQL e Redis com portas apenas em `127.0.0.1`; API e Workers rodam na máquina com `npm run dev:api` e `npm run start:worker`. O app aponta para a máquina de desenvolvimento por `EXPO_PUBLIC_API_URL` (IP da rede local; HTTP só em desenvolvimento). Dados de teste nunca vêm de dados reais de alunos.

15.2 Staging

A VM gratuita é uma só. Um ambiente de staging na mesma VM (`docker compose -p aprovaai-staging`, outro subdomínio, outro banco e outro Redis, limites menores) divide recursos com produção e só deve existir se for necessário, com os limites de memória reduzidos. As VMs AMD micro (1 GB) não comportam Postgres e Redis juntos com folga. O perfil `preview` do EAS pode apontar para o staging.

16. Escala e migração para o Render

16.1 Gatilhos para migrar

Qualquer um destes, mantido por uma semana, pede a migração (valores a calibrar): CPU `p95` da VM acima de 70%, memória acima de 80%, atraso de filas violando os alertas do §13.4, falta de capacidade

ou recuperação da VM pela Oracle, ou o custo de operação superar o custo de pagar. A migração é decisão de custo e precisa de aprovação.

16.2 Mapeamento

Hoje (VM)	No Render	Preço (30/10/2026)
API	Serviço web	US$ 7/mês (menos de 1 CPU, 512 MB)
PostgreSQL	Postgres gerenciado	US$ 6/mês (256 MB de RAM, 100 conexões, 1 GB de SSD)
Redis	Key Value	US$ 10/mês (256 MB, persistência em disco só em planos pagos); confirmar se a política `noeviction` é configurável †
Workers e relay	Serviços em segundo plano	Preço não verificado nesta consulta
Caddy e certificados	Incluídos no serviço web	—

Total confirmado: **US$ 23 por mês** (≈ US$ 0,023 por usuário/mês a cada 1.000 usuários). Se o Worker e o relay forem cobrados como o serviço web (US$ 7 cada), o total fica em cerca de US$ 37 por mês (estimativa, a confirmar); para economizar, o relay pode rodar no mesmo processo do Worker no Render (muda a composição em `bootstrap/`, não as camadas). Os planos gratuitos do Render **não** servem: o Key Value gratuito tem 25 MB e sem persistência em disco, incompatível com as exigências do BullMQ (§7.1), e o Upstash gratuito (500 mil comandos por mês) é consumido pelo polling do BullMQ.

16.3 Passos da migração

1. Provisionar os serviços no Render e configurar as mesmas variáveis (nenhum valor sai do cofre da equipe para o repositório).
2. Janela de manutenção curta: parar a API, gerar um `pg_dump` final e restaurá-lo no Postgres do Render.
3. Subir os serviços; o reconciliador republica o que estava em processamento (o Redis novo começa vazio).
4. Reduzir o TTL do DNS de `api` antes e apontar o registro para o Render; validar HTTPS e `/readyz`.
5. Manter a VM desligada por uma semana antes de remover.

17. Custos e limites gratuitos por mil usuários

Recurso	Custo hoje	Observação
Oracle Always Free	US$ 0	Dentro dos limites do §3.2; risco de ociosidade (§4.3)
GitHub (Actions, repositório, pacotes)	US$ 0	Repositório público
EAS Build	US$ 0	15 builds por plataforma por mês; Starter (US$ 19) se faltar
EAS Update	US$ 0 até 1.000 usuários ativos por mês	Opcional
Sentry	US$ 0	5 mil erros por mês; Team US$ 29

Recurso	Custo hoje	Observação
Grafana Cloud	US$ 0	10 mil séries e 50 GB de logs; Pro a partir de US$ 19
HTTPS, DNS e domínio	US$ 0	Domínio existente; Let's Encrypt †
Contas Apple e Google Play	Fora do escopo	Já existem (D12)
IA (GPT 6.1 - Terra)	**US$ 34 por mês a cada 1.000 assinantes Premium** (teto, cenário base, §1.8)	Único custo que cresce por usuário; US$ 28,30 com prefixo em cache e US$ 54 com saída de 2.000 tokens
Render (se migrar)	US$ 23 a cerca de US$ 37 por mês	§16.2

O custo de infraestrutura é praticamente **fixo** nesta escala: o que cresce com usuários é a IA, que depende da fração de assinantes Premium (com 10% de Premium, ≈ US$ 3,40 por mês a cada 1.000 usuários). Pontos de atenção que podem gerar cobrança e exigem aviso antes: passar de 1.000 usuários ativos com EAS Update, esgotar 15 builds, estourar 5 mil erros no Sentry, 10 mil séries ou 50 GB de logs no Grafana, e qualquer migração para o Render. A cobrança do Premium (comissões de loja e eventual RevenueCat) está em aberto e descrita no §1.8.

18. Runbooks

Cada runbook começa por confirmar o alerta e termina por registrar a causa e a duração.

R1. Deploy normal. Aprovar o fluxo de promoção; acompanhar `/readyz`; verificar painéis de erros e filas por 15 minutos.

R2. Rollback. Rodar a promoção com o `<sha>` anterior; em emergência, `IMAGE_TAG=<sha> docker compose up -d` na VM. Dados: ver R3.

R3. Restaurar o PostgreSQL. Parar API, Workers e relay; descriptografar o backup com a chave privada da equipe; restaurar com `pg_restore` num banco novo; validar contagens; trocar o banco; subir os serviços; o reconciliador republica o que estava em processamento. Registrar a duração (compara com o RTO de 2 horas).

R4. Reprocessar `dead-letter`. Investigar a causa (log por `correlationId`); corrigir; usar o script administrativo do Back-End para republicar as mensagens no outbox com `jobId` novo; acompanhar a drenagem. Nunca por rota pública.

R5. Redis cheio ou com AOF corrompido. Memória acima de 70%: verificar a retenção de jobs concluídos e falhos (§9.2 do Back-End) e a profundidade das filas. AOF corrompido: `redis-check-aof --fix` num backup do arquivo; se falhar, subir um Redis vazio e deixar o reconciliador republicar.

R6. VM recuperada, perdida ou corrompida. Criar nova VM A1 (§4); rodar `bootstrap-host.sh`; clonar `infra/`; restaurar os segredos do cofre da equipe; restaurar o último backup (R3); atualizar o registro `A` de `api` se o IP mudou (TTL curto ajuda); validar HTTPS e `/readyz`. Meta: cerca de 2 horas.

R7. HTTPS ou certificado com falha. Ver o log do Caddy; confirmar o DNS e as portas 80 e 443; se houve emissões repetidas com erro, aguardar o limite de taxa e usar o ambiente de testes da Let's Encrypt para depurar.

R8. IA indisponível, `429` ou orçamento estourado. Confirmar o alerta de falha da IA; as redações ficam na fila e são repetidas pelo backoff. Se o orçamento mensal estourou, o freio pausa a fila (as

redações **não** falham): decidir com o responsável pelo produto aumentar o orçamento ou aguardar o mês seguinte. Registrar o consumo real.

R9. Projeções atrasadas. Ver a fila `projections` e a idade do job mais antigo; reiniciar o Worker; se perdeu mensagens, rodar o script de reconstrução de projeções.

R10. Segredo vazado. Revogar e trocar imediatamente (chave da IA, `JWT_SECRET`, senhas, token do EAS); reiniciar os serviços; revisar logs de acesso; se dados de alunos foram expostos, acionar o responsável pela LGPD (decisão em aberto no §14 da arquitetura). Se o segredo entrou no repositório público, trocá-lo mesmo que o commit seja apagado: o histórico é copiável.

R11. Fila de redação parada. Ver o Worker (`docker compose logs worker`), a conexão com o Redis e o limite de taxa da IA; reiniciar o Worker (o `worker.close()` evita jobs travados); conferir o outbox.

19. Ordem de implementação e critérios de aceite

Os marcos `F#` e `B#` referem-se aos guias do Front-End e do Back-End.

Marco	Escopo	Critério observável	Depende de
D0	Preparação: `.env` no `.gitignore`; política de segredos; proteção de ramos; contas e região da Oracle decididas	`git check-ignore .env` confirma o arquivo ignorado; ramos protegidos nas configurações do GitHub	—
D1	CI de app e backend; Dependabot	O fluxo `ci` fica verde em pull request e reprova com erro proposital de lint	F0, B0
D2	VM da Oracle: provisionamento e endurecimento	SSH só por chave e só dos IPs da equipe; portas 80 e 443 acessíveis de fora; `bootstrap-host.sh` repete sem erro	D0
D3	Compose de produção, DNS e HTTPS	`https://api.<seu-domínio>/readyz` responde `200` por fora; Postgres e Redis inacessíveis de fora; AOF e `noeviction` confirmados com `redis-cli config get`	D2, B1
D4	Entrega contínua do backend e rollback	Uma versão sobe pelo fluxo de promoção e um rollback para o `<sha>` anterior funciona, ambos medidos	D3, B2
D5	Observabilidade e alertas	Painéis do §13.3 ativos; um alerta provocado de propósito (por exemplo, derrubar o Worker) chega por e-mail	D3, B6
D6	Backups e simulado de restauração	Um backup criptografado é restaurado num contêiner descartável com a duração registrada	D3
D7	Builds móveis e lojas	Build `production` instalado em aparelho real; envios a TestFlight e ao teste interno do Google Play	F8, D3

20. Checklist de go-live e pendências

20.1 Antes de abrir para usuários

- Todos os marcos D0 a D7 e B0 a B7 concluídos, com evidências registradas.
- Segredos fora do repositório; `git log` e o histórico sem segredos; `.env` ignorado.
- HTTPS válido, `/metrics` fora do proxy, Postgres e Redis inacessíveis de fora (verificado por varredura externa).
- Alertas ativos e testados; runbooks revisados por quem estará de plantão.
- Backup de ontem restaurável (simulado feito no mês).
- Limites de custo conhecidos: contagens de usuários ativos, builds, erros, séries e logs acompanhadas; custo da IA medido em semanas reais.
- LGPD: base legal, retenção, exclusão, política de privacidade e consentimento definidos pelo produto.
- Fonte do Premium decidida; sem isso, nenhum usuário recebe o plano em produção.

20.2 Pendências que bloqueiam trabalho

Pendência	Quem decide	Bloqueia
Região de origem e criação da conta Oracle; eventual upgrade para Pay As You Go	Responsável pelo produto	D2
Aprovação das propostas P9 (observabilidade) e P10 (entrega)	Responsável pelo produto	D1, D4, D5
Domínio para a API (`api.<seu-domínio>`) e o contato do certificado	Responsável pelo domínio	D3
Destino externo gratuito dos backups	Equipe de DevOps	D6
Confirmação da isenção de ociosidade em conta Pay As You Go	Equipe, com o suporte da Oracle	Avaliação de risco (§4.3)
Preço de Workers e Key Value configurável no Render	Equipe, na consulta ao provedor	Migração (§16)
`bundleIdentifier` , `package` , política de privacidade e conta de teste das lojas	Produto	D7
Fonte, preço e comissões do Premium; LGPD	Produto e jurídico	Go-live